

LAB 01

Title: Binary Search Using C++ with Debugging Techniques.

Objective

To learn how to debug a C++ Binary Search program using Breakpoints and the Watch Window, and to understand how the Binary Search algorithm finds a target element in a sorted array.

Procedure

1. Complete environment setup.
2. Write the Binary Search code.
3. Set break points.

Breakpoint Location	Reason
while(low <= high)	To observe each iteration of the Binary Search loop.
int mid = low + (high - low) / 2;	To verify the calculated middle index.
if(arr[mid] == target)	To check whether the target element has been found.
else if(arr[mid] < target)	To observe when the search moves to the right half of the array.
else	To observe when the search moves to the left half of the array.

4. Execute with Next Line.
5. Track the changes on Watch.

Code with break-points

```

BinarySearch.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int arr[n];
10
11     for(int i = 0; i < n; i++)
12         cin >> arr[i];
13
14     int target;
15     cout << "Enter Target: ";
16     cin >> target;
17
18     int low = 0;
19     int high = n - 1;
20     int mid;
21
22     while(low <= high)

```

```
23 {  
24     mid = low + (high - low) / 2;  
25  
26     if(arr[mid] == target)  
27     {  
28         cout << "Element found at index " << mid << endl;  
29         return 0;  
30     }  
31     else if(arr[mid] < target)  
32     {  
33         low = mid + 1;  
34     }  
35     else  
36     {  
37         high = mid - 1;  
38     }  
39 }  
40  
41 cout << "Element not found";  
42  
43 return 0;  
44 }
```

Output

The screenshot displays a C++ IDE with three main panes. The left pane, titled 'Debug', shows the state of variables during execution: `low = 3`, `high = 4`, `mid = 3`, `arr = {3, 4, 5, 8, 9}`, `arr[mid] = 8`, and `target = 8`. The middle pane shows the source code for `BinarySearch.cpp`, with line 28 (`cout << "Element found at index " << mid << endl;`) highlighted. The right pane shows the output of the program, which includes the prompt `Enter Target: 8` and the result `Element found at index 3`. The bottom of the IDE features a toolbar with buttons for `Debug`, `Add watch`, `Next line`, `Continue`, `Stop Execution`, `View CPU window`, `Into function`, and `Skip function`.

Title: Selection Sort Using C++ with Debugging Techniques

Objective

To learn how to debug a C++ Selection Sort program using Breakpoints and the Watch Window, and to understand how Selection Sort arranges elements in ascending order.

Procedure

1. Complete environment setup.
2. Write the Selection Sort code.
3. Set break points.

Breakpoint Location	Reason
for(int i = 0; i < n - 1; i++)	To observe the beginning of each pass of the Selection Sort algorithm.
int minIndex = i;	To verify the initial minimum element for the current pass.
if(arr[j] < arr[minIndex])	To determine when a new minimum element is found.
swap(arr[i], arr[minIndex]);	To observe the swapping of the current element with the minimum element.

4. Execute with Next Line.
5. Track the changes on Watch.

Code with break-points

```

SelectionSort.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int arr[n];
10
11     for(int i = 0; i < n; i++)
12         cin >> arr[i];
13
14     for(int i = 0; i < n - 1; i++)
15     {
16         int minIndex = i;
17
18         for(int j = i + 1; j < n; j++)
19         {
20             if(arr[j] < arr[minIndex])
21             {
22                 minIndex = j;
23             }
24         }
25     }

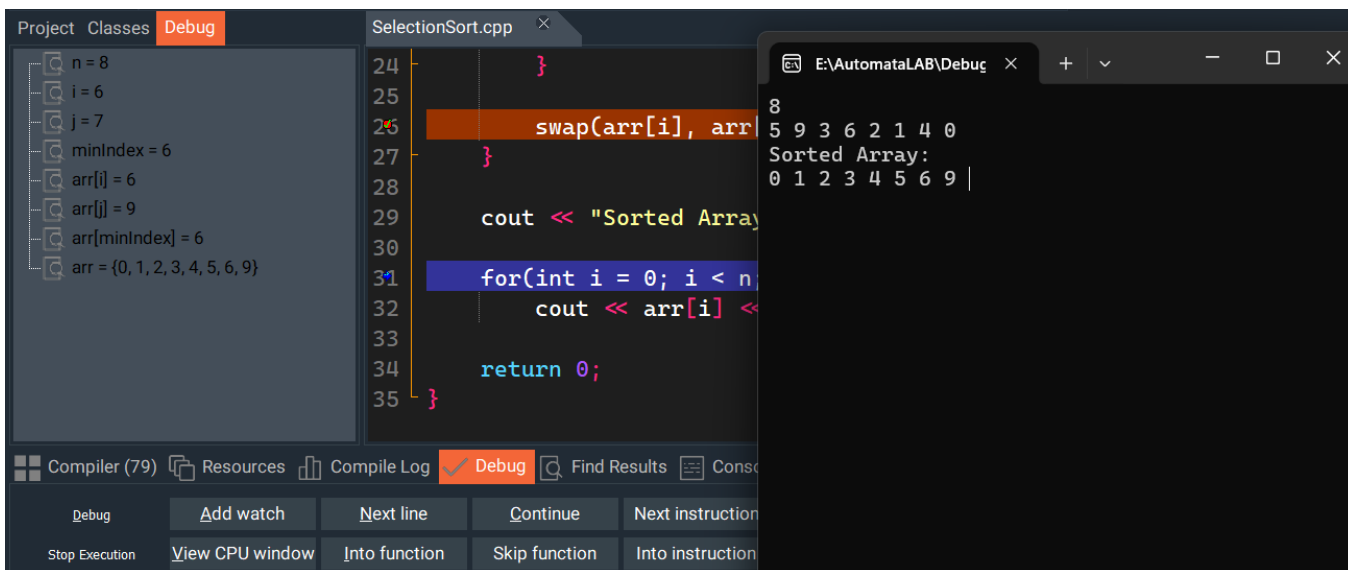
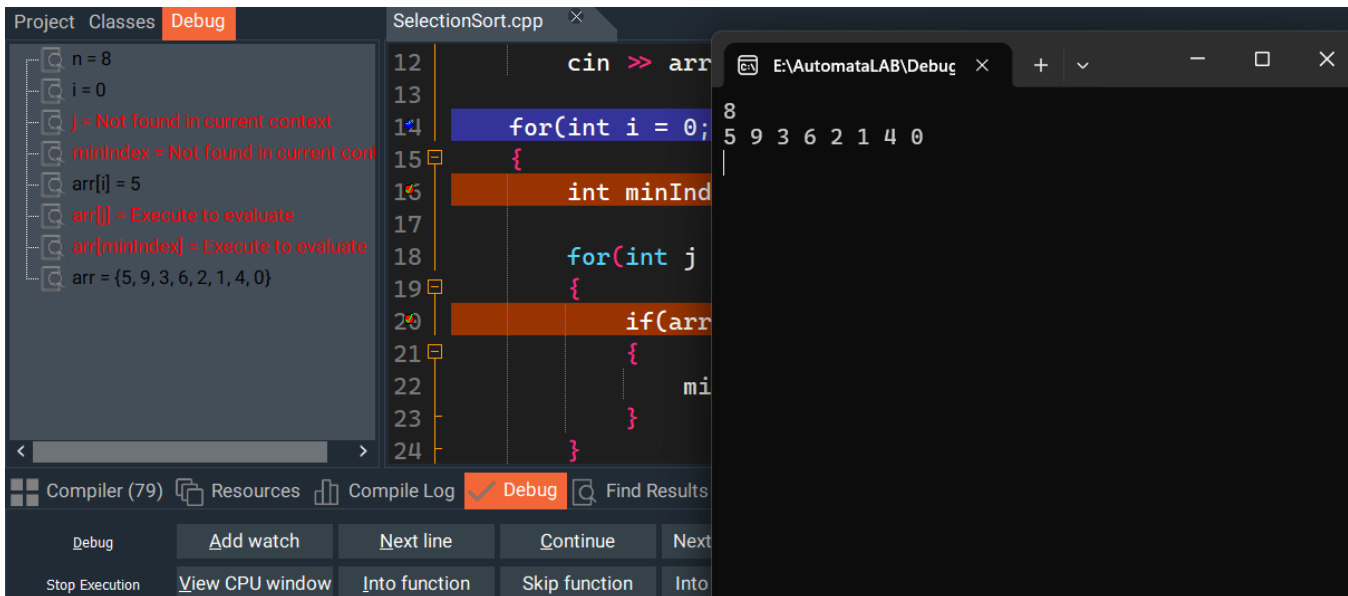
```

```

25 swap(arr[i], arr[minIndex]);
26 }
27
28
29 cout << "Sorted Array:" << endl;
30
31 for(int i = 0; i < n; i++)
32     cout << arr[i] << " ";
33
34 return 0;
35 }

```

Output



Title: Matrix Multiplication Using C++ with Debugging Techniques

Objective

To learn how to debug a C++ Matrix Multiplication program using Breakpoints and the Watch Window, and to understand how two matrices are multiplied using nested loops.

Procedure

1. Complete environment setup.
2. Write the Matrix Multiplication code.
3. Set break points.

Breakpoint Location	Reason
result[i][j] = 0;	To verify that each element of the result matrix is initialized before multiplication.
for(int k = 0; k < c1; k++)	To observe each multiplication step for a matrix element.
result[i][j] += A[i][k] * B[k][j];	To track how the value of each element in the result matrix is accumulated.
cout << result[i][j] << " ";	To verify the final value before printing each element.

4. Execute with Next Line.
5. Track the changes on Watch.

Code with break-points

```

MatrixMultiplication.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int r1, c1, r2, c2;
7
8      cout << "Enter rows and columns of the first matrix: ";
9      cin >> r1 >> c1;
10
11     int A[10][10];
12
13     cout << "Enter elements of the first matrix:" << endl;
14     for(int i = 0; i < r1; i++)
15         for(int j = 0; j < c1; j++)
16             cin >> A[i][j];
17
18     cout << "Enter rows and columns of the second matrix: ";
19     cin >> r2 >> c2;
20
21     int B[10][10];
22
23     cout << "Enter elements of the second matrix:" << endl;
24     for(int i = 0; i < r2; i++)
25         for(int j = 0; j < c2; j++)

```

```

26     cin >> B[i][j];
27
28     if(c1 != r2)
29     {
30         cout << "Matrix multiplication is not possible." << endl;
31         return 0;
32     }
33
34     int result[10][10];
35
36     for(int i = 0; i < r1; i++)
37     {
38         for(int j = 0; j < c2; j++)
39         {
40             result[i][j] = 0;
41
42             for(int k = 0; k < c1; k++)
43             {
44                 result[i][j] += A[i][k] * B[k][j];
45             }
46         }
47     }
48
49     cout << "\nResult Matrix:" << endl;

```

```

50
51     for(int i = 0; i < r1; i++)
52     {
53         for(int j = 0; j < c2; j++)
54             cout << result[i][j] << " ";
55
56         cout << endl;
57     }
58
59     return 0;
60 }

```

Output

The screenshot displays a C++ IDE with the following components:

- Project Explorer (Left):** Shows a tree of variables:
 - A[10][10] = 2
 - r1 = 2
 - c1 = 2
 - i = 1
 - j = 1
 - A[i][j] = 5
 - A[i][k] = 5
 - B[k][j] = 9
 - result[i][j] = 73
- Source Code (Middle):** Shows the code from line 50 to 60, with the current execution point at line 59 (return 0;).
- Output Window (Right):** Displays the program's output:


```

Enter rows and columns of the first matrix: 2
2
Enter elements of the first matrix:
2 3
4 5
Enter rows and columns of the second matrix:
2 2
Enter elements of the second matrix:
6 7
8 9
Result Matrix:
36 41
64 73

```